

Лабораторна робота №2 “Масиви та списки”

Мета – дослідити масиви та списки, основні операції з даними за допомогою таких структур даних.

Теоретичні відомості

Одновимірні масиви

Потреба у масивах виникає тоді, коли в оперативній пам'яті зберігається велика, але визначена кількість однотипних даних.

Наприклад, можна задати масив щоденних значень температури повітря протягом місяця з метою визначення середньої температури або масив логічних значень, що зображуватиме наявність квитків на кіносеанс на всі місця у кінозалі.

Розрізняють *одновимірні* та *багатовимірні* масиви. Зокрема, масив температур є одновимірним, а масив квитків - двовимірним.

Масив – структурний тип, який об'єднує в собі набір елементів деякого типу, об'єднаних спільним ім'ям (ім'ям масиву).

Поняття масиву та його властивості

Одновимірний масив — це послідовність однотипних даних. Уважно проаналізувавши це означення, можна зробити висновок, що масив фактично поєднує в собі дві структури: множину елементів і заданий на цій множині порядок.

Усі елементи масиву мають один і той самий тип, що називається *базовим*.

З іншого боку, порядок теж визначається набором значень одного й того самого типу, що називається *індексним*, а самі ці значення називаються *індексами*.

Кожному елементу масиву відповідає певний індекс. Індексний тип має бути простим порядковим типом даних.

Кількість елементів в одновимірному масиві називається його *розмірністю*, або *довжиною*.

Тип масиву — це структурований тип даних, множина допустимих значень котрого складається з усіх масивів, для яких зафіксовано:

- розмірність;
- базовий тип;
- індексний тип;
- множину значень індексу.

Наведене визначення типу масиву можна застосувати до типів як одновимірних, так і багатовимірних масивів.

Усі елементи одновимірного масиву записуються до розташованих поряд ділянок оперативної пам'яті. Тому і весь масив може розглядатися як одна нерозривна область пам'яті.

Загальний вигляд створення масиву: вказуємо

- ім'я масиву;
- тип елементів;
- кількість елементів.

Одномірні масиви ще називають *лінійними*.

Нумерація елементів у масиві (як і в рядках) починається з 0-го.

Довжина масива (кількість елементів) визначається полем `.length` (на відміну від методу `.length()` для рядків).

Недоліки використання масивів:

- В неупорядкованому масиві пошук виконується повільно.
 - В упорядкованому масиві повільно виконується вставлення.
 - Видалення виконується повільно, як в упорядкованому так і в неупорядкованому масиві.
 - Неможливо змінити розмір масиву після його створення.
 - звичайна змінна може містити **лише 1 значення**.
 - масив – це проста змінна, яка може **містити не одне, а відразу кілька значень**, на відміну від звичайної змінної, що містить єдине значення.
- Масив – це такий собі "магічний" контейнер, у який ми можемо покласти відразу кілька значень ("коробочок").

Нижче наводимо Вам приклади масивів.

Приклад №1

`int [ ] array1 = { 5, 17, 350 }`

тип даних масиву квадратні дужки вказують, що це масив ім'я масиву в фігурних дужках вказані числа, які ми присвоїли в array1

Як бачите, ми створили цілочисельний масив, який назвали `array1` і присвоїли три числа: 5, 17, 350

Приклад №2

`String [ ] array2 = { "Java", "is the best" }`

тип даних масиву квадратні дужки вказують, що це масив ім'я масиву в фігурних дужках вказані слова / фрази, які ми присвоїли в array2

Як бачите, ми створили рядковий масив, який назвали array2 і присвоїли слово "Java" і фразу "is the best". Причому ми написали їх у подвійних лапках, тому що це масив типу String.

Будь-який масив має довжину

Наприклад, довжина цього масиву = 3. Тому що в масиві 3 числа: 5, 17, 350

`int [ ] array1 = { 5, 17, 350 }`

тип даних масиву квадратні дужки вказують, що це масив ім'я масиву в фігурних дужках вказані числа, які ми присвоїли в array1

А довжина цього масиву = 2. Тому що в масиві 2 значення: "Java" і "is the best"

`String [ ] array2 = { "Java", "is the best" }`

тип даних масиву квадратні дужки вказують, що це масив ім'я масиву в фігурних дужках вказані слова / фрази, які ми присвоїли в array2

Якщо сказати простіше, то довжина масиву – це кількість елементів, які в ньому зберігаються (кількість маленьких "коробочок" у великій).

Пропонуємо тепер Вам відповісти, яка довжина цих 3 масивів?

```
int [ ] array3 = { 1, 8, 15, 30, 2, 3 } ;
```

```
byte [ ] array5 = { 1, 3 } ;
```

```
String [ ] citiesArray = { "Київ", "Львів", "Луцьк", "Одеса" } ;
```

Правильні відповіді:

```
int [ ] array3 = { 1, 8, 15, 30, 2, 3 } ;
```

Довжина цього масиву 6. Тому що в цьому масиві 6 чисел: 1, 8, 15, 30, 2, 3

```
byte [ ] array5 = { 1, 3 } ;
```

Довжина цього масиву 2. Тому що в масиві 2 числа: 1 і 3

```
String [ ] citiesArray = { "Київ", "Львів", "Луцьк", "Одеса" } ;
```

Довжина цього масиву 4. Тому що в масиві 4 слова: "Київ", "Львів", "Луцьк", "Одеса".

- Підраховувати вручну кількість елементів у масиві – це добре, але якщо елементів дуже багато ... Навряд чи у Вас вистачить терпіння підрахувати 3 000 елементів. Якщо використовувати властивість масиву `length`, довжина масиву буде підрахована автоматично.
- Наприклад, у нас є ось такий масив.

`int [ ] array1 = { 5, 17, 350 }`

↑
тип даних
масиву

↑
квадратні дужки
вказують, що
це масив

↑
ім'я масиву

↑
в фігурних дужках
вказані числа, які
ми присвоїли в `array1`

-
- Пишемо `int k = array1.length;` .
- У результаті створиться змінна `k`, у якій буде знаходитися значення 3.

- Спробуйте запустити ось цей код, щоб переконатися в цьому на практиці:

```

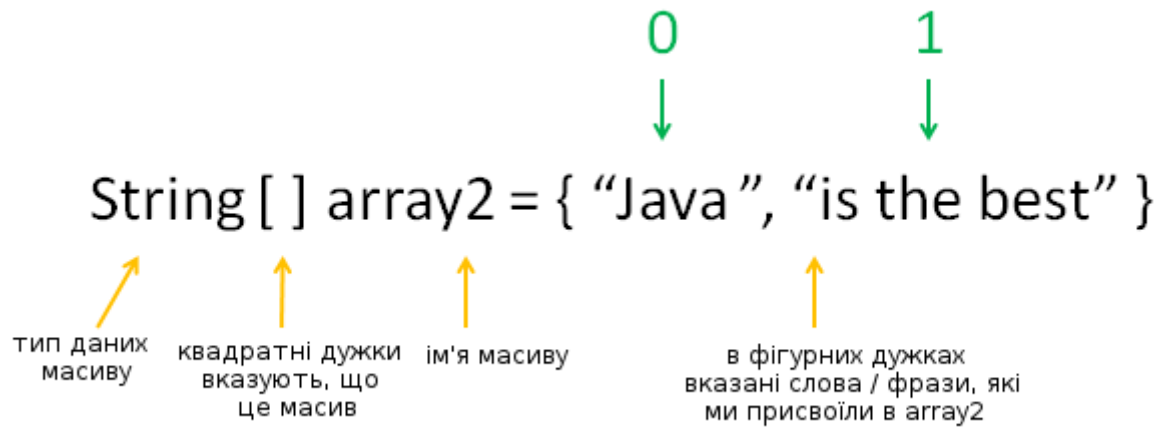
• class Test {
•
• public static void main(String[] args) {
• int [] array1 = {5,17,350};
• int k = array1.length;
• System.out.println(k);
• }
• }

```

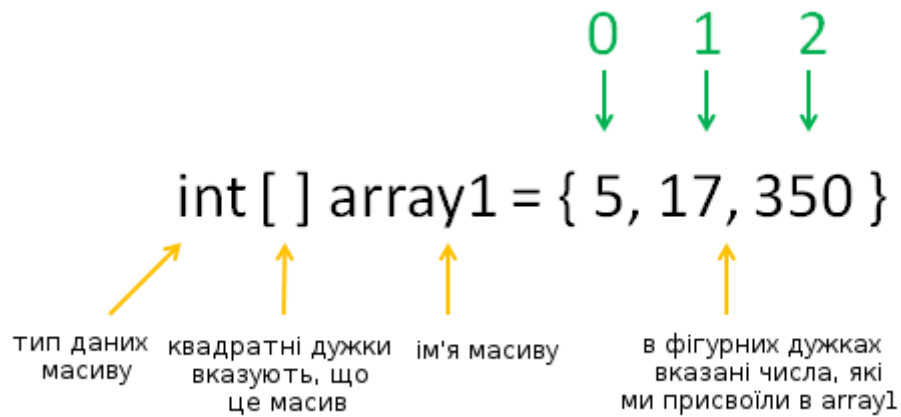
-
- У будь-якому масиві значення елементів зберігаються під порядковими номерами (індексами)
- Причому нумерація починається з "нуля". Пропонуємо розглянути на прикладах.
- Приклад №1
- У цьому масиві:
- 5 – це елемент з порядковим номером 0
- 17 – це елемент з порядковим номером 1
- 350 – це елемент з порядковим номером 2



-
- Приклад №2
- У цьому масиві:
- "Java " - це елемент з порядковим номером 0
- "is the best" - це елемент з порядковим номером 1



-
- Якщо Ви дуже любите солодощі, то можете уявити масив у вигляді макарунів.
-
- Тобто індекс елемента – це його номер по порядку "мінус" 1.
- Як видно, довжина масиву з макарунів = 3. І застосовується та ж логіка до порядкових номерів (індексів) у масиві.
- Розберемося, навіщо в масиві значення елементів зберігаються під порядковими номерами (індексами)?
- А відповідь дуже і дуже проста. Щоб можна було звертатися до конкретних елементів масиву за індексом. Оскільки з того, що ми в велику коробку хаотично кидаємо інші коробки, толку мало. Нам потрібно знати, де лежить маленька коробка з тим, що нам потрібно. Тому все, що ми кладемо у велику коробку, має свій номер.
- Для того, щоб отримати те, що лежить у маленькій коробочці, нам потрібно звернутися до неї за індексом. Для цього пишемо назву змінної (велика коробка), після чого без пробілів у квадратних дужках вказуємо індекс коробки.
- масив[номер]
- Розглянемо це на практиці.
- Як ми можемо звернутися до елемента масиву array1 під індексом 0? Відповідь: array1[0]
- Як ми можемо звернутися до елемента масиву array1 під індексом 1? Відповідь: array1[1]
- Як ми можемо звернутися до елемента масиву array1 під індексом 2? Відповідь: array1[2]



-
- А як створити пустий масив?
-
- У всіх попередніх прикладах ми створювали масив і відразу ініціалізували його якимись значеннями. Однак є можливість спочатку створити масив потрібного розміру, а пізніше наповнювати його значеннями (якщо нам при створенні невідомо, які значення там знаходитимуться). Наприклад, лікар каже жінці, що в неї буде двійня, але, якої статі будуть діти, поки не відомо. Ось Вам масив на 2 елементи типу "дитина", але значення поки не відомі.
- Щоб створити такий масив, потрібно скористатися ключовим словом `new`. Тепер створення масиву буде виглядати так:
- `тип [] назва = new тип[розмір]; [об]`
- Запам'ятайте, що типдо "дорівнює" і після, якщо ми говоримо про масив з примітивних типів, повинен бути однаковим!
- У майбутньому ми побачимо, що це правило може порушуватися, але це вже інша історія.
- У програмі це виглядало б ось так:

```

• class Test {
•
• public static void main(String[] args) {
•
• int [] array1 = new int[3];
•
• }
• }

```

- Розберемо цей рядок коду:

```

• int[] array1=new int[3];

```

- За допомогою `int[] array1` ми вказали, що буде цілочисельний масив з ім'ям `array1`
- За допомогою `= new int[3];` ми створили порожній масив з 3 елементів.

- Зверніть увагу: незважаючи на те, що масив array1 вважається порожнім, насправді в ньому все-таки є значення за замовчуванням... Під час створення порожнього масиву елементів присвоюються значення залежно від типів даних:
- для int - 0
- для float, double - 0.0
- для String - значення null
- для char - \0
- для boolean - значення false
- Оскільки в нас у наведеному прикладі масив array1 цілочисельний, це значить, що всі 3 елементи в ньому дорівнюють нулю.. Тобто `int [] array1 = new int [3];` це те ж саме, що і `int [] array1 = {0,0,0}`
- А як наповнити такий масив значеннями?
- Спробуйте запустити цей код на своєму комп'ютері:

```

• class Test {
•
• public static void main(String[] args) {
•
• int[] array1 = new int[3];
•
• array1[0]=5;
• array1[1]=17;
• array1[2]=350;
•
• System.out.println(array1[0]);
• System.out.println(array1[1]);
• System.out.println(array1[2]);
• }
• }

```

- За допомогою ось цих рядків ми звертаємося до кожного елемента масиву і наповнюємо його потрібним значенням:

```

• array1[0]=5;
• array1[1]=17;

```

- `array1[2]=350;`

- Тобто, якщо `array1 [0]` стоїть праворуч від "дорівнює", то ми отримуємо його значення, якщо зліва – присвоюємо йому значення.
- А ці рядки коду ми додали, щоб Ви могли побачити, що таки так, кожному елементу масиву були присвоєні значення 5, 17, 350 відповідно.

- `System.out.println(array1[0]);`
- `System.out.println(array1[1]);`
- `System.out.println(array1[2]);`

- У консолі Ви побачите:
- 5
- 17
- 350
- Ще один важливий момент – розмір створеного масиву не можна змінити
- Разом подивимося на ось цей рядок коду:

- `int[] array1;`

- Як Ви думаєте, цим рядком коду ми створили масив? Відповідь – ні. Чому? Адже ми начебто:
- вказали за допомогою `[]`, що це масив
- вказали за допомогою `int`, що масив цілочисельний
- вказали, що назва масиву `array1`
- А масив не створений ??? Так, масив не створений.
- Масив створюється, коли під нього виділяється пам'ять. А виділяється пам'ять під масив тільки в момент виконання наступної сходинки:

- `int[] array1 = new int[3];`

- тобто коли ми дописали `= new int [3];`
- або цієї:

- `int[] array1 = {1,2,3}`

- Ви не можете додавати елементи в масив понад зазначену кількість, а не заповнені вами осередки просто будуть зберігати значення "0". У Java є інші інструменти, які дозволяють робити "безрозмірні" списки елементом. З ними Ви познайомитеся пізніше.

-

- Підсумуємо:
- 1. Масив – це змінна, у яку можна покласти не одне, а відразу кілька значень.
- 2. Усі елементи масиву мають однаковий тип.

- 3. При створенні порожнього масиву елементів присвоюються значення залежно від типу даних масиву:
 - для `int` - 0
 - для `float`, `double` - 0.0
 - для `String` - значення `null`
 - для `char` - `\0`
 - для `boolean` - значення `false`
- 4. Розмір масиву не можна змінити після його створення.
-
- Зв'язані списки – структури даних, які вирішують вищезгадані проблеми. Крім того, зв'язаний список може бути основою для створення інших структур даних (таких як стеки та черги).
- Зв'язані списки можна використовувати замість масивів (окрім ситуацій з частим випадковим доступом до окремих елементів по індексу).
- Зв'язані списки не вирішують усіх проблем збереження даних, але вони універсальні та прості на концептуальному рівні.
- Однозв'язний список - це структура даних, в якій кожен елемент даних, міститься у спеціальному об'єкті – елементі списку (класу, на основі якого створюються ці об'єкти, часто надається ім'я `Link`).
- Кожен об'єкт цього класу містить посилання на наступний елемент списку(поле `next`).

Приклади реалізації масивів та списків:

- Оскільки масив потребує виділення динамічної пам'яті, то у класичному випадку треба використовувати команду `new`, аналогічно випадку з типом `String`.
- Оголошення масиву:
 - `тип[] ім'я = new тип[розмір];` `тип[] ім'я = {ел0, ел1, ..., елN};`
- Приклади оголошення та створення масивів:
 -
 - `int[] a;`
 - `double[] ar1;`
 - `double ar2[];`
 - `int[] mas1 = {10,20,30};`
 - `int[] mas2 = new int[3];`
 - `int [ ] a = new int [2];` // масив з 2-х цілих елементів, ім'я масиву – `a`
 - `int a [ ] = new int [2];` // і так можна
 - `String [ ]sm = new String[10];` // масив рядків
 - `a = new int[10];` // масив із 10 елементів типу `int`

- `int n = 5;`
- `ar1 = new double[n];` // масив із 5 елементів `double`
- `ar2 = {3.14, 2.71, 0, -2.5, 99.123};` // масив із 6 елементів типу `double`
-
- У Java передбачено створення масиву одночасно з ініціалізацією, з якої впливає і розмір масиву
- `int [ ] mas = {10, 20, 30, 40, 50}`
-
- Наприклад, так можна вивести на екран елементи будь-якого масиву з ім'ям `ar2`:
-
- `for(int i = 0; i <= ar2.length - 1; i++) {`
- `System.out.print(ar2[i] + " ");`
- `}`
- Для стислості зручніше змінити нестрогу нерівність на строгу, тоді не потрібно буде віднімати одиницю з розміру масиву. Заповнимо масив цілими числами від 0 до 9 і введемо його на екран:
- `for(int i = 0; i < ar1.length; i++) {ar1[i] = Math.floor(Math.random() * 10);`
- `System.out.print(ar1[i] + " ");`
- `}`

Зверніть увагу, на кожному кроці циклу ми спочатку відправляємо випадкове значення в елемент масиву з `i`-м індексом, а потім цей же елемент виводимо на екран. Але два процеси (наповнення і виведення) можна було зробити і в різних циклах. наприклад:

```
for(int i = 0; i < ar1.length; i++)
{ ar1[i] = Math.floor(Math.random() * 9);
}
for(int i = 0; i < ar1.length; i++) {
    System.out.print(ar1[i] + " ");
}
```

В даному випадку більш раціональний перший спосіб (один прохід по масиву замість двох), але не завжди можливо виконати необхідні дії в одному циклі. Для обробки масивів завжди використовуються цикли типу «n раз» (`for`) тому, що нам заздалегідь відомо скільки разів повинен повторитися цикл (стільки ж разів, скільки елементів у масиві).

Обробка лінійного масиву

Дано масив з 10 випадкових цілих чисел, кожне з яких – з проміжку (-20; 20).

1. Вивести на екран усі від'ємні елементи.
2. Знайти суму додатних елементів.

3. Підрахувати кількість нульових елементів.

Розв'язання. У програмі спочатку ініціалізуємо масив випадковими числами, відповідно до умови задачі. Потім покажемо масив на екрані. Це допоможе перевірити правильність виконання обрахунків. Виконаємо обчислення та відобразимо їх.

Лістинг

```
package prog3;
public class Prog3 {
    public static void main(String [ ]args) {
        int [ ] a = new int[10];
        // ініціалізація масиву випадковими цілими
        for (int i=0; i<a.length; i++) { a[i]=(int)(Math.random()*40-20); }
        // виведення вхідного масиву на екран
        System.out.println("Вхідний масив:");
        for (int i=0; i<a.length; i++) { System.out.println("a["+i+"]="+a[i]); }
        // будемо рахувати суму додатних - змінна S
        int S = 0;
        // для кількості нулів виділимо окрему змінну
        int k = 0;
        System.out.println("Від'ємні елементи:");
        // проглядаємо кожен елемент масиву
        for (int i=0; i<a.length; i++) {
            // якщо елемент від'ємний, показуємо його
            if (a[i]<0) System.out.println("a["+i+"]="+a[i]);
            // інакше, якщо додатній - збільшуємо суму S
            else if (a[i]>0) S=S+a[i];
            // в іншому випадку (тобто коли a[i]==0)
            else k++;
        }
        System.out.println("Сума додатних: "+S);
        System.out.println("Кількість нульових: "+k);
    }
}
```

Приклад результатів роботи програми:

Вхідний масив:

```
a[0]=10
a[1]=10
a[2]=12
a[3]=16
a[4]=1
a[5]=-16
a[6]=-1
a[7]=17
a[8]=9
```

a[9]=2

Від'ємні елементи:

a[5]=-16

a[6]=-1

Сума додатних: 77

Кількість нульових: 0

Знаходження максимуму та мінімуму в масиві

Дано: масив дійсних чисел. **Знайти:** Максимальний(max) та мінімальний(min) елементи масиву.

ЗМІННІ:

Вхідні:

масив із десяти додатних та від'ємних дійсних чисел.

Вихідні:

max - максимальний елемент масиву;
min - мінімальний елемент масиву.

Лістинг

```
public class maxMin {  
    public static void main(String[] args) {  
        int a[] = new int[]{1, 4, -1, 2, 5, 10, 7, 8, 9, -10 };  
        int max=0;  
        int min =0;  
        for(int i=0; i<a.length; i++)  
            System.out.print(a[i]+" ");  
        System.out.println();  
        for(int i=0; i<a.length; i++)  
            if(a[i]>max){  
                max=a[i];  
            }  
            else{  
                if(a[i]<min){  
                    min=a[i];  
                }  
            }  
        System.out.println(max);  
        System.out.println(min);  
    }  
}
```

Пошук максимуму та мінімуму та обмін місцями

Дано: масив дійсних чисел.

Знайти: Максимальний (max) та мінімальний (min) елементи масиву, поміняти їх місцями та вивести новий масив на консоль.

Вхідні: масив із десяти додатних та від'ємних дійсних чисел, що не повторюються

Вихідні: масив відповідно виконаному завданню.

Лістинг

```
public static void main(String[] args) {
    int a[] = new int[]{1, 4, -1, 2, 5, 10, 7, 8, 9, -10};
    int max=0;
    int min=0;
    int imax = 0;
    int imin = 0;
    for(int i=0; i<a.length; i++)
        System.out.print(a[i]+"");
    System.out.println();

    for(int i=0; i<a.length; i++){
        if(a[i]>max){
            max=a[i];
            imax=i;
        }
        if(a[i]<min){
            min=a[i];
            imin=i;
        }
    }

    a[imin] = max;
    a[imax] = min;
    for(int i=0; i<a.length; i++)
        System.out.print(a[i]+"");
    System.out.println();

    System.out.println(max);
    System.out.println(min);
}
```

Результат

Вхідний масив:	1	4	-1	2	5	10	7	8	9	-10
Вихідний масив:	1	4	-1	2	5	-10	7	8	9	10

max=10

min=-10

Особливості реалізації зв'язаного списку

Розглянемо початок реалізації класу Link:

```
class Link {  
    public int iData; // інформація  
    public double dData; // інформація  
    public Link next; // посилання на наступний елемент списку  
  
    public Link(int iData, double dData) {  
        this.iData = iData;  
        this.dData = dData;  
    }  
  
    public void displayLink() {  
        System.out.print(" {" + iData + ", " + dData + "}");  
    }  
}
```

Створимо поля, які буду нести певну інформацію (iData, dData) та поле (next), яке має посилання на об'єкт того ж типу (такі поля також називають самовідносними (self-referential)).

Клас Link зберігає не об'єкт, а лише посилання на нього. Посилання - це число, яке асоціюється з об'єктом. Іншими словами, посилання – адреса об'єкту в пам'яті. На відміну від масиву, де потрібний елемент можна знайти за індексом, знайти елемент у списку можна лише одним способом – відслідкувати його по ланцюжку елементів від початку списку.

Клас Link буде містити лише один метод, відображення даних конкретного вузла.

```
public void displayLink() {  
    System.out.print(" {" + iData + ", " + dData + "}");  
}
```

Новий клас, який буде «об'єднанням» наших елементів, безпосередньо буде реалізацією зв'язаного списку. Назвемо цей клас LinkedList. Він буде містити наступні методи:

1. insertFirst - метод, який вставляє новий елемент списку на першу позицію

Реалізація методу:

```
public void insertFirst(int iD, double dD) {  
    Link newLink = new Link(iD, dD); //створюємо новий елемент  
    newLink.next = first; //вказали на старий "перший" елемент і  
    first = newLink; // позначили створений елемент, як перший  
}
```

2. find – метод, який забезпечує пошук інформації у нашому списку

```
public Link find(int key) { // пошук елемента с заданим ключем  
    Link current = first; // починаємо пошук з першого елемента  
    while (current.iData != key) { // якщо, ключ не знайдено,то..  
        if (current.next == null) { //якщо ключа немає в списку взагалі,  
            return null;  
        } else {  
            current = current.next; //то переходимо до наступного  
        }  
    }  
    return current;  
}
```

3. delete – метод, який забезпечує видалення певного елемента списку

Пошук елемента аналогічний розглянутому вище. Під час видалення елемента, потрібно розглянути декілька ситуацій. По перше, якщо елемент який видаляється займає першу позицію, то ми просто зміщуємо посилання First, на другий елемент в списку, а про перший «забуваємо». По друге – якщо елемент який видаляється знаходиться всередині списку, то його потрібно обійти.

Реалізація:

```
public Link delete(int key) { //видалення елемента по заданому ключу
    Link current = first;
    Link previous = first;

    while (current.next != null) { // пошук елемента
        if (current.next == null) {
            return null; // елемент не знайдено
        } else {
            previous = current; // перейти до наступного
            current = current.next;
        }
        // ключ знайдено
        if (current == first) {
            first = first.next; // якщо шуканий елемент -
            // перший
        } else {
            previous.next = current.next; // якщо шуканий
            // елемент лежить всередині списку, обійдемо його
        }
    }
    return current;
}
```

4. displayList метод відображення даних у нашому списку

```
public void displayList() {
    Link current = first;
    System.out.print("List (first --> last): ");

    while(current != null) { //доки список не закінчився,
        // відображаємо дані
        current.displayLink(); // відображаємо дані елемента, на
        // якому знаходимося
        current = current.next; // переходимо до наступного
        // елемента
    }
    System.out.println("");
}
```

Основні методи списку реалізовані. Необхідно також реалізувати обробку помилки пошуку неіснуючого елемента та метод видалення останнього елемента списку.

Лістинг:

```
class Link {
    public int iData; // інформація
    public double dData; // інформація
    public Link next; // посилання на наступний елемент списку

    public Link(int iData, double dData) {
        this.iData = iData;
        this.dData = dData;
    }

    public void displayLink() {
        System.out.print(" {" + iData + ", " + dData + "}");
    }
}

class LinkedList {
    Link first;

    public LinkedList() {
        first = null;
    }

    public void insertFirst(int iD, double dD) {
        Link newLink = new Link(iD, dD); //створюємо новий елемент
        //списку
        newLink.next = first; //вказали на старий "перший" елемент і
        //таким чином зробили його другим
        first = newLink; // позначили створений елемент, як перший
    }

    public Link find(int key) { // пошук елемента с заданим ключем
        Link current = first; // починаємо пошук з першого елемента
        while (current.iData != key) { // якщо, ключ не знайдено,то..
            if (current.next == null) { //якщо ключа немає в списку взагалі,
                //повертаємо null
                return null;
            } else {
                current = current.next; //то переходимо до наступного
                //елемента і шукаємо ключ
            }
        }
    }
}
```

```

        return current;
    }

    public Link delete(int key) { //видалення елемента по заданому ключу
        Link current = first;
        Link previous = first;

        while (current.next != null) { // пошук елемента
            if (current.next == null) {
                return null; // елемент не знайдено
            } else {
                previous = current; // перейти до наступного
                current = current.next;
            }
            // ключ знайдено
            if (current == first) {
                first = first.next; // якщо шуканий елемент -
                // перший елемент лежить всередині списку, обійдемо його
            } else {
                previous.next = current.next; // якщо шуканий
            }
        }
        return current;
    }

    public void displayList() {
        Link current = first;
        System.out.print("List (first --> last): ");

        while(current != null) { //доки список не закінчився,
            // відображаємо дані
            current.displayLink(); // відображаємо дані елемента, на
            // якому знаходимося
            current = current.next; // переходимо до наступного
            // елемента
        }
        System.out.println("");
    }

}

public class LinkedList {
    public static void main(String[] args) {
        LinkedList newList = new LinkedList();
    }
}

```

```

        newList.insertFirst(12, 30.99);
        newList.insertFirst(10, 30.99);
        newList.insertFirst(45, 345.34);
        newList.insertFirst(14, 23.78);
        newList.insertFirst(54, 12.59);

        newList.displayList();

        Link f = newList.find(10);
        System.out.println("Ми знайшли: " + f.iData);
    }
}

```

Варіанти виконання лабораторної роботи:

Номер в списку	Задача зі структурою даних масиви	Задача зі структурою даних - списки
1	Визначити максимальний елемент та видалити його	Сформувати список з елементом, що має нульове значення та видалити його
2	Визначити мінімальний елемент та видалити його	Сформувати список дійсних чисел без нульового елемента та додати нульовий елемент
3	Реалізувати помилку пошуку неіснуючого елемента	Знайти мінімальний і максимальний елементи та поміняти їх місцями
4	Знайти мінімальний і максимальний елементи та поміняти їх місцями	Визначити максимальний елемент та видалити його
5	Знайти парні числа в масиві дійсних чисел та сформувати новий масив	Визначити максимальний елемент та видалити його
6	Сформувати масив випадкових чисел та підрахувати нульові елементи	Реалізувати помилку пошуку неіснуючого елемента
7	Сформувати масив випадкових чисел та підрахувати елементи більше одиниці	Визначити парні та непарні номери списку та поміняти їх значення місцями

8	Визначити максимальний елемент та подвоїти його	Сформувати список з елементом, що має нульове значення та видалити його
9	Визначити мінімальний елемент та зменшити його на 1	Сформувати список дійсних чисел без нульового елемента та додати нульовий елемент
10	Реалізувати помилку пошуку неіснуючого елемента	Знайти мінімальний і максимальний елементи та знайти їх суму
11	Знайти мінімальний і максимальний елементи та знайти їх добуток	Визначити максимальний елемент та додати до нього 18
12	Знайти парні числа в масиві дійсних чисел та сформувати новий масив з суми елементів, що стоять поряд	Визначити максимальний елемент та додати до нього 7
13	Сформувати масив випадкових чисел та підрахувати елементи, значення яких більше 4	Реалізувати помилку пошуку неіснуючого елемента
14	Сформувати масив випадкових чисел та підрахувати елементи більше 12	Визначити парні та непарні номери списку та сформувати масив зі значеннями суми парного та непарного елемента

Студенти, що мають номер в списку більше 14, беруть варіант за другою цифрою.

Можна реалізувати задачу з курсової або власну задачу.

Візуалізація за власним прикладом.

Висновки – приклади використання списків та масивів.